

Federated PHC Supply Chain Platform — 10-Day Build Guide (Updated)

Team of 4. Native Android app. Agentic AI via Google AI Studio (Gemini). Core loop: prescription entry → demand forecasting (own PHC + nearby PHCs) → stock-out/expiry flagging → human-approved cross-PHC restock request. Plus a visibility dashboard with an impact counter, an early-warning layer, an emergency/surge mode, and a federated architecture story for the BRICS-scale ask.

Roles

- **A — Android:** Kotlin/Compose app, offline-first data layer (Room), background sync, multi-language UI (dynamic translation), one-tap approval UI
 - **B — Backend/data:** FastAPI, PostgreSQL/TimescaleDB, schema (incl. expiry, optional patient_ref), synthetic multi-PHC dataset, nearby-PHC lookup
 - **C — Forecasting:** demand model (own + nearby-PHC weighted), stock-out + expiry logic, early-warning spike detection, surge-mode behavior
 - **D — Agents/orchestration:** Gemini function-calling agents (with reasoning output) on AI Studio, dashboard incl. impact counter, demo/pitch
-

Phase 0 — Day 1: Foundations & planning

Everyone works together before splitting off.

- Lock data model: phc(id, name, district, lat, lon), drug(id, name), stock(phc_id, drug_id, quantity, expiry_date, updated_at), prescription(id, phc_id, drug_id, qty, doctor_id, patient_ref[optional: age_band, gender], timestamp), transfer_request(id, from_phc, to_phc, drug_id, qty, reason[shortage|expiry], status, approved_by)
- Agree API contract between Android ↔ backend ↔ agents
- Decide the nearby-PHC radius and weighting rule (e.g. 15km, 1/distance or 3-tier bucket) — used by both forecasting and procurement agent
- Confirm patient data scope: no EHR, anonymous prescriptions, optional non-identifying patient_ref only
- Set up repo, branches; single AI Studio API key held by D, usage tracking sheet started
- Pick the demo story: district, # PHCs, the stock-out scenario, and the surge-mode scenario
- Environments running for all 4

Checklist — Phase 0

- ☐ Data model agreed (incl. expiry, patient_ref, transfer reason)
 - ☐ API contract drafted
 - ☐ Nearby-PHC radius/weighting rule decided
 - ☐ Patient data scope confirmed (no EHR)
 - ☐ Repo/branches set up; AI Studio key owned + usage sheet started
 - ☐ Demo scenario chosen, incl. a surge-mode moment
 - ☐ All 4 environments running
-

Phase 1 — Days 2-3: Parallel core builds (mocked data/agents only)

Build and test against stubs — no real Gemini calls yet.

A — Android

- Prescription entry (offline Room DB, background sync worker)
- Language layer scaffolded from the start: keep the app on a single English string set, wire in a translation API (ML Kit / Google Translate) with a language picker, so any local language is a config addition rather than new build work

B — Backend/data

- CRUD endpoints for stock (with expiry), prescriptions, transfer requests
- `get_nearby_phcs(phc_id, radius)` — shared utility for both forecasting and procurement
- Synthetic dataset: 8-12 PHCs, seasonal demand, a planted stock-out, a planted near-expiry surplus, a planted district-wide spike (for early-warning + surge demo)

C — Forecasting

- Baseline time-series model per drug per PHC
- Nearby-PHC weighted signal added from the start (own trend + distance-weighted neighbor trend)
- Stock-out threshold logic (predicted demand vs. stock vs. lead time)
- Expiry-flag logic (surplus nearing expiry → eligible for transfer)
- Early-warning spike detector across a PHC's catchment

D — Agents/orchestration

- Write Gemini function-calling schemas for all 3 agents, each returning a short reasoning field alongside its action — test against mocked responses
- Sketch dashboard layout: stock, beds, staff, alerts, and an impact-counter placeholder

Checklist — Phase 1

- ☐ Android: offline prescription entry works; string resources scaffolded for 2 languages
 - ☐ Backend: endpoints correct against mocks; nearby-PHC lookup working
 - ☐ Synthetic dataset includes stock-out, near-expiry, and spike scenarios
 - ☐ Forecasting: nearby-PHC-weighted prediction works on synthetic data
 - ☐ Expiry-flag logic working on synthetic data
 - ☐ Agent schemas (incl. reasoning field) tested against mocks — no real API calls yet
 - ☐ Dashboard wireframe incl. impact-counter placeholder agreed
-

Phase 2 — Days 4-5: Deepen each track

- **A:** sync conflict handling, PHC/doctor login, one-tap approve/reject UI for incoming transfer recommendations, request-status view
- **B:** deploy backend; add bed-availability and staff-attendance tables (simple count + timestamp per PHC); add `approved_by/status` flow to `transfer_request`
- **C:** validate forecasts on held-out data, tune thresholds for the

demo scenario, build the surge-mode parameter (e.g. a multiplier applied to forecast demand + widened redistribution radius)

- **D:** first real, limited Gemini calls — validate function-calling incl. reasoning output end-to-end on a few cases, then back to mocks; start the dashboard's impact-counter logic (reads from transfer_request log)

Checklist — Phase 2

- ☐ Android: one-tap approve/reject flow working; sync conflicts handled
 - ☐ Backend deployed; bed/staff data flowing; approval status tracked
 - ☐ Surge-mode parameter implemented in forecasting
 - ☐ Real Gemini calls validated on a handful of cases (reasoning field present), then back to mocked responses
 - ☐ Impact-counter logic reading real transfer data
 - ☐ Daily AI Studio usage checked against budget
-

Phase 3 — Days 6-7: Integration

- Wire Android → backend → forecasting → agents → dashboard as one live pipeline
- Real Gemini calls only at real triggers (prescription submitted, nightly forecast, threshold breach) — never on every UI refresh
- Add response cache/replay layer for repeated dev/testing
- Wire human-in-the-loop: agent recommends → officer approves on Android → transfer executes → dashboard/impact counter updates
- Wire expiry-based triggers alongside shortage triggers, both feeding the same procurement agent path
- Wire surge mode end-to-end: toggling it visibly changes forecasts and redistribution radius live
- Federated/BRICS layer: implement as a config toggle showing "district model" vs. "shared model," with only aggregated stats/weights crossing the boundary — real or clearly-labeled architectural mock
- Full test of the planted scenarios: stock-out loop, expiry-redistribution loop, spike/early-warning, surge mode

Checklist — Phase 3

- ☐ Full pipeline runs end-to-end on all planted scenarios without manual patching
 - ☐ Human approval step wired and working on Android
 - ☐ Expiry-triggered and shortage-triggered transfers both working through the same agent path
 - ☐ Surge mode visibly changes system behavior live
 - ☐ Reasoning trail visible in the UI/dashboard for agent decisions
 - ☐ Federated/BRICS layer represented (real or clearly-labeled mock)
 - ☐ Impact counter and dashboard reflect live data across all simulated PHCs
-

Phase 4 — Days 8-9: Hardening & credit management

- Fix integration bugs from Phase 3
- Run the full demo scenario (incl. surge mode) 3-5 times end-to-end to confirm repeatability
- Freeze real API usage from here on — only during scheduled rehearsals; all other dev/testing uses cached/replayed responses
- Verify translated UI across the languages you'll demo (check for text overflow/truncation, since translated strings run longer or

shorter than English), polish Android UI and dashboard (incl. impact counter styling)

- Prepare a fallback (recorded run / cached responses) in case of live API issues during the demo

Checklist — Phase 4

- ☐ Demo scenario (incl. surge mode) passes 3+ consecutive runs
 - ☐ All non-rehearsal dev work uses cached/mock responses only
 - ☐ Multi-language UI verified across demo languages, no layout breakage
 - ☐ Fallback plan ready
 - ☐ Usage log reviewed — comfortable headroom before demo day
-

Phase 5 — Day 10: Demo & pitch

- Final rehearsal (1–2 live runs max) plus dry runs on cached data
- Pitch narrative: problem → working node (live demo, incl. surge mode) → federated/BRICS extension (architecture) → impact counter as the "so what"
- Be upfront about what's real vs. simulated
- Assign presenters (suggest: A demos the Android app incl. approval flow, D drives agents/dashboard/surge mode, C explains forecasting + early-warning, B fields architecture/scale questions)

Checklist — Phase 5

- ☐ Final rehearsal completed, timing checked
 - ☐ Pitch narrative written and assigned
 - ☐ Clear framing of live vs. simulated vs. architectural
 - ☐ Fallback (cached run) ready as backup
-

AI Studio credit / rate-limit plan (applies throughout)

1. One key, one owner (D), usage tracked daily
2. Mock first — real calls only from Phase 3 onward
3. Trigger-based calls only, never polling
4. Cache and replay real responses for repeated dev/testing and dry runs
5. Flash-tier model for agent orchestration; save heavier models only if a step truly needs it
6. Batch multi-PHC/multi-drug checks instead of looping calls
7. Fix the exact live-call count for demo day in advance and rehearse that count